# Interaction Scaling: Grounding the Third Axis of Test-Time Compute

Bojie Li
Pine AI

Noah Shi
University of Washington

**Abstract**

There are two standard ways to spend more compute at test time: let a model *reason* longer, or *sample* more attempts and keep one. Both share a hidden limit: they are *internal*. Every extra token comes from the same frozen weights and the same prompt, so neither can tell the model anything it does not already know. We study a third way, *interaction*: the model proposes an artifact, an external instrument observes how it actually behaves, and the model revises. Each cycle imports a real observation, so interaction breaks through the ceiling the other two hit.

We argue that a single variable governs this third axis, **grounding**, and that it must hold on *both* sides of the loop. The feedback that drives revision must come from an instrument that actually observes the flaw, and so must the metric that scores the result. On hard coding tasks at a fixed token budget, reasoning-only and best-of-$N$ sampling both plateau (the latter even when an oracle picks the best sample), while every interaction strategy keeps improving; our proposer–reviewer harness reaches a perfect 100% pass rate with no run-to-run variance, and the gain holds across three model families. On rendered visual artifacts, the usual judge (a vision–language model, or VLM, reading a screenshot) rates 14 of 15 visibly broken figures "perfect," because the screenshot hides the flaws before the judge can see them. A tool that measures the real layout instead shows the loop removing 40–74% of defects across four modalities; and that same VLM, used as the *reviewer*, makes slide layouts worse where the measuring tool repairs them. Interaction scaling is real and distinct from reasoning and sampling, but only visible when both the feedback and the metric are grounded.

Code: https://github.com/19PINE-AI/interaction-scaling
Website: https://01.me/research/interaction-scaling

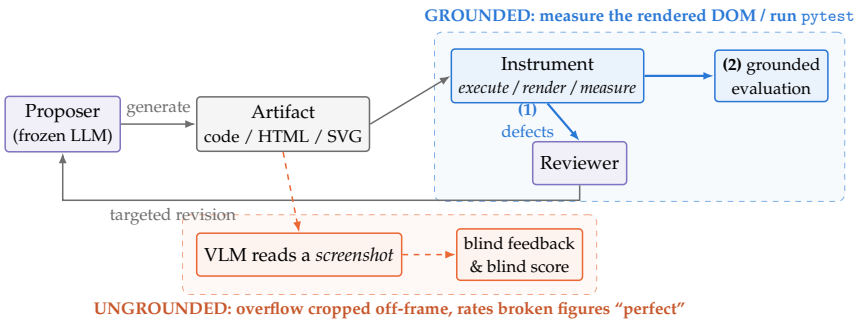



**Figure 1. Grounding must hold on both sides of the interaction loop.** One instrument observation is both **(1)** grounded *feedback* (the defect list that drives revision and escapes the internal ceiling) and **(2)** grounded *evaluation*, the score that makes the gain measurable. The default VLM-on-a-screenshot judge (orange lane) breaks both: the screenshot drops the defects before the model sees them.

## 1 Introduction

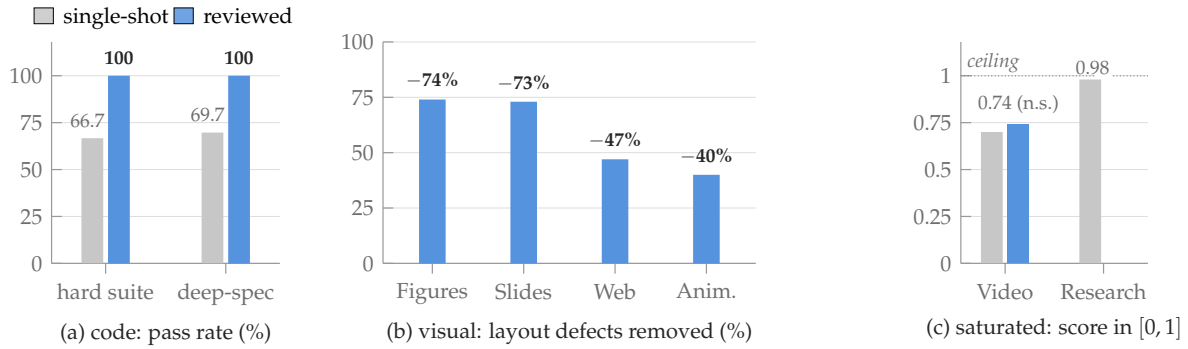
Once a model is trained, the way we make it better on a hard task is no longer to add parameters or data (*training-time scaling*) but to spend more compute per query on the frozen model: *inference-time scaling* (OpenAI, 2024; Snell et al., 2025). On the artifact-producing tasks we study (code, web pages, slides, figures, animations, video edits, research reports), this is the lever we can still pull. Prior work formalizes two ways to pull it. *Reasoning scaling* (Wei et al., 2022; Yao et al., 2023a; DeepSeek-AI, 2025; Snell et al., 2025) spends more tokens thinking before committing; *sampling scaling* (Wang et al., 2023b; Brown et al., 2024) draws more attempts and selects one. They look different, but share a property this paper treats as fundamental: both are **internal**. Every extra token, whether in a longer chain of thought or another sample, comes from the same frozen weights and the same fixed prompt. More internal compute reshuffles what the model already has; it imports nothing new.

We study a third form of inference-time scaling that breaks out of this closed loop: *interaction scaling*, in which the model queries an external instrument that observes the artifact itself. The model runs the tests, renders the page and measures the layout, or issues the search query and reads the result. Such an observation is not a function of the weights; it reports how the artifact actually behaves, and can carry information the model never had. This axis is well established in practice (Shinn et al., 2023; Yao et al., 2023b; Gou et al., 2024; Shen et al., 2025), but accounts of *why* and *when* it beats internal scaling remain informal and, we argue, only half-stated. Industry practice shows the stakes: in a controlled 900-run deployment study at ByteDance, frontier coding models passed the functional-correctness bar yet fell short on delivery quality until a feedback "harness" was added, and the study had to invent a multi-axis quality metric before the gap was even visible (Hong, 2026). That is a field sighting of exactly the two-sided problem we formalize below (Section 8).

**Grounding is the variable, on both sides of the loop.** Our thesis is that one variable governs interaction scaling: **grounding**. Feedback is *grounded* when it comes from an instrument observing the artifact's actual form or behavior, not from a model voicing an opinion about it. And grounding must hold on *both* sides of the proposer–reviewer loop (Figure 1):

1. **Grounded feedback** (the signal that drives revision): run the code and read the failing test, render the HTML and measure where each element lands, issue the query and read the result. Prior work focuses on this side; a short information argument (Section 2) explains why it lets interaction break the internal ceiling.
2. **Grounded evaluation** (the metric that *measures* the gain): the same observation, used as the score. This side is almost always overlooked, and getting it wrong hides the effect entirely.

The second point matters more than it may seem. The standard evaluator for visual artifacts is a vision–language model (VLM) reading a screenshot, and a screenshot *drops the flaw before the judge ever sees it*: overflowing content is cropped off-frame, and small overlaps fall below the model's visual acuity. On dense academic figures this judge rates 14 of 15 single-shot renders "perfect," where a direct measurement finds only 3 actually clean (Section 6); used as the *reviewer*, the same VLM makes slide layouts worse (Section 5). Ungrounded on either side, the third axis either does not fire or cannot be seen.



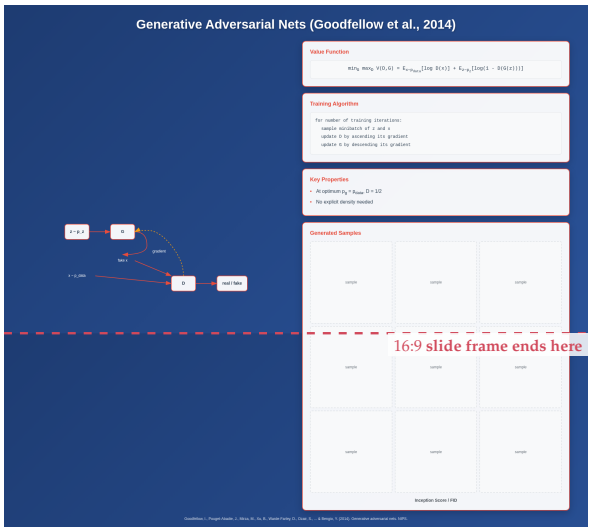

(a) code: pass rate (%) (b) visual: layout defects removed (%) (c) saturated: score in $[0, 1]$

**Figure 2. Results at a glance: one grounded loop, seven modalities. (a)** Execution feedback lifts both code suites to a strict 100% pass rate, recovering every single-shot failure (3-seed means; Section 4). **(b)** Grounded geometry feedback removes 40–74% of the layout defects a deterministic DOM instrument measures, on all four visual modalities (every reduction statistically decisive; Section 6). **(c)** The two remaining modalities are scoped negatives: video editing is already strong single-shot (the reviewed lift is not significant), and deep research saturates at 0.98 single-shot factual accuracy, leaving no headroom for feedback to claim (Section 6).

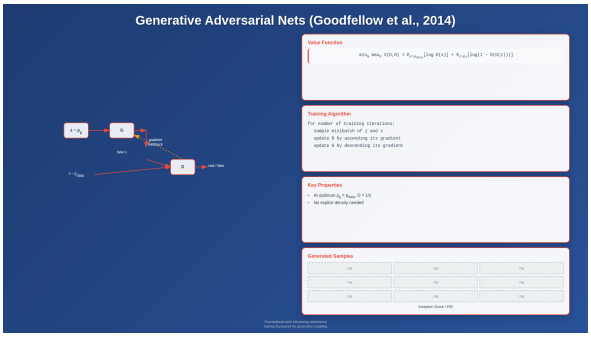
**What the loop delivers, at a glance.** Figure 2 summarizes what one frozen model in one harness achieves across the seven modalities we test. Grounded execution feedback lifts both hard code suites to a perfect 100% pass rate; grounded geometry feedback removes 40–74% of the measured layout defects on four visual modalities; and the two modalities that leave no single-shot *headroom* (no room to improve before the metric's ceiling) are reported as honest negatives, not manufactured wins. Figure 3 makes this concrete on one dense-slide task: the defects the loop removes are exactly the ones the standard screenshot judge cannot see.

**Contributions.**

1. **A grounding framework (Section 2).** We sort feedback into two kinds (*ungrounded* model opinion vs. *grounded* instrument observation) and add a *coverage* principle: grounded feedback helps exactly as far as its instrument can observe. A short information argument (formalized in Appendix A) explains why internal scaling saturates, and the same argument shows that an ungrounded *metric* cannot measure the gain. Beyond the known *biases* of model-as-judge evaluation (Zheng et al., 2023), we pin down a sharper failure: where quality is a measurable physical property, a screenshot judge is *structurally blind*, not merely noisy.
2. **Interaction keeps scaling where internal compute stops (Section 4).** At the same token budget, reasoning-only and best-of-$N$ saturate (the latter even with an oracle verifier), while every interaction strategy climbs toward a perfect pass rate; the proposer–reviewer harness does so at the lowest token cost and with zero seed variance.
3. **The feedback instrument must observe the defect (Section 5).** Swapping only the reviewer's instrument on a fixed suite: on behavioral bugs, execution feedback converges far more cheaply than critique; a linter (grounded, but observing only surface form) buys nothing; and a screenshot-reading VLM reviewer makes slide geometry *worse*, where a geometry reviewer repairs it.
4. **So must the metric (Section 6).** The standard VLM judge passes 14 of 15 broken figures; a




**(a) Single-shot** (6 measured defects): the content overflows the 16:9 slide frame (dashed line) by more than half the frame's height: exactly the defect class a screenshot judge never sees, because the capture crops at the frame.




**(b) After grounded feedback** (0 defects): the same task after the propose→measure→revise loop: the sample grid is compacted and the panels rebalanced so every element sits inside the frame.

**Figure 3. A concrete example of what the loop fixes.** A dense real-paper slide task (the GAN paper), rendered full-page at equal width: single-shot (a) vs. reviewed (b). The deterministic geometry instrument counts 6 defects single-shot and 0 after review (the same 6 → 0 reduction replicates in a second seed), and a screenshot judge sees none of them (Section 6).

tool that measures the rendered layout instead reveals large, statistically decisive defect reductions across four visual modalities, gains the standard metric cannot see.

## 2 A Framework for Internal and External Test-Time Compute

### 2.1 Internal vs. external test-time compute

All three axes we consider are forms of *inference-time scaling*: on a frozen model, they spend more compute per query rather than adding parameters or data (OpenAI, 2024; Snell et al., 2025). What separates them is where the extra compute draws its information from. Reasoning and sampling differ in mechanics, but not in kind. A longer chain of thought re-derives consequences of what the weights and prompt already contain; best-of-$N$ re-draws from the same distribution and picks one. In both, every token comes from the same two sources (frozen weights, fixed prompt), so we call them **internal** scaling. **External** scaling is different in kind: the artifact is handed to an instrument outside the model (a test runner, a layout engine, a search index), and the instrument's observation is fed back into the next generation. That observation reflects the *artifact's actual behavior*, not the model's beliefs about it.

### 2.2 A feedback taxonomy, and the coverage principle

We classify a reviewer's feedback channel by *who produces the signal* (Figure 4):

- **Ungrounded feedback** is a model's opinion: an LLM critiques its own artifact, or a VLM rates a screenshot. The screenshot case is worth care. A screenshot *is* an instrument observation, but a lossy one that drops exactly the defects at issue (content off the canvas,

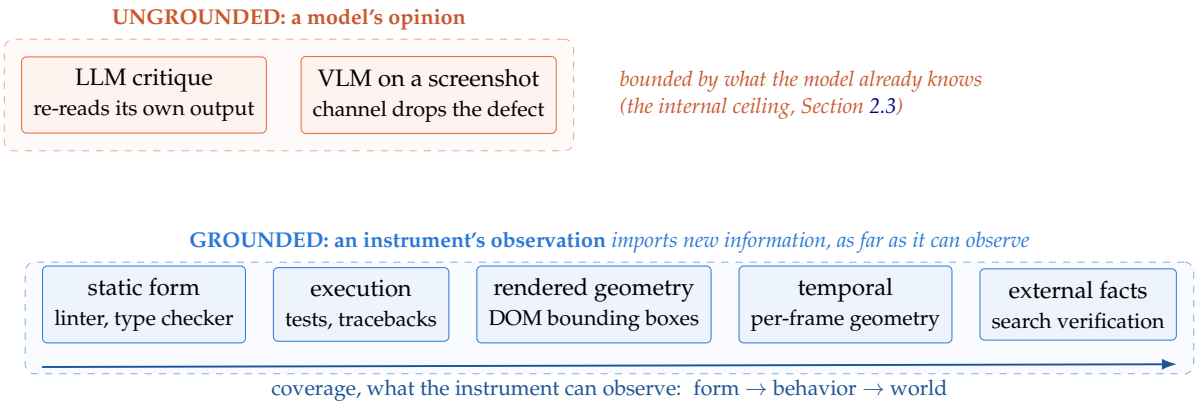



**Figure 4. The feedback taxonomy.** Feedback is either a model's opinion (*ungrounded*, top) or an instrument's observation of the artifact (*grounded*, bottom); grounded subtypes are named by what the instrument observes, and the *coverage* axis orders them by reach. The screenshot-fed VLM sits in the ungrounded lane because the signal that enters the loop is a model's reading of an already-lossy view. Ungrounded feedback is subject to the internal ceiling (Section 2.3); grounded feedback escapes it, but only for defects inside the instrument's coverage: a linter cannot see a runtime bug, and a screenshot cannot see off-canvas overflow.

overlaps too small to see), so the signal that enters the loop is the model's reading of those pixels. We classify a channel by what really produces its signal: here, the model.

- **Grounded feedback** is an instrument's observation, its subtypes named by *what the instrument observes*: **static form** (linters, type checkers inspect the source), **execution behavior** (tests report the failing assertion), **rendered geometry** (a layout engine reports every element's true bounding box), **temporal behavior** (the same, per animation frame), and **external facts** (a search engine checks a claim against the world).

Grounding alone, however, is not sufficient, and this is the second half of the taxonomy:

> **The coverage principle.** A feedback channel helps exactly as far as its instrument's observational reach. A linter is grounded but observes only form, so it cannot see a runtime bug; a screenshot is an observation whose channel crops the defect out, so a VLM judging it cannot see broken geometry. The instrument must *observe the property that is broken*.

The coverage principle is what unifies the paper's results: it predicts that execution feedback beats critique on behavioral bugs while a linter does not (Section 5), that a screenshot-fed VLM fails both as a reviewer (Section 5) and as a judge (Section 6), and that a deterministic geometry instrument succeeds at both.

### 2.3 Why internal scaling saturates

The argument is one sentence long: *a model re-reading its own output cannot learn anything it did not already know.* A critique from the same weights that produced the artifact is post-processing, which adds no information about the correct answer; best-of-$N$ can only *select* among the candidates the model actually draws, so even a perfect verifier cannot return a program it

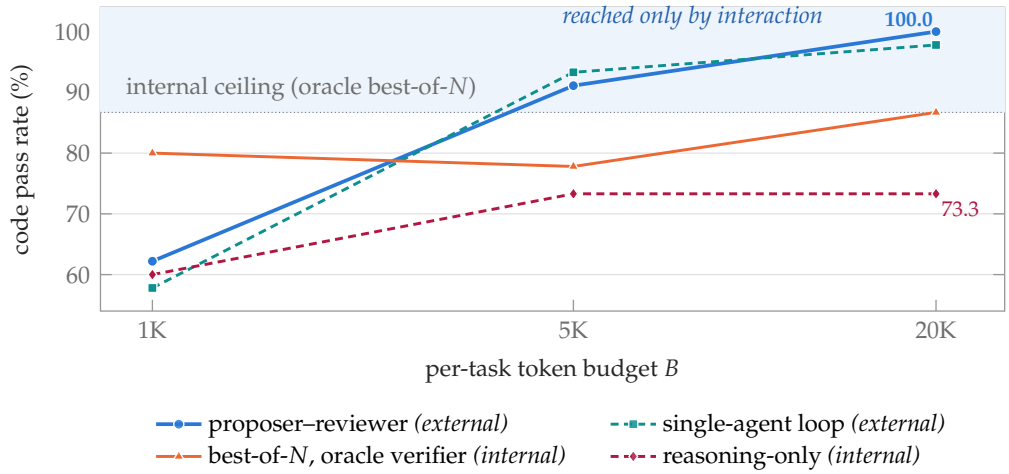



**Figure 5. Internal scaling saturates; external scaling does not (Prediction 1).** Code pass rate vs. per-task token budget on the 15 hard tasks (3-seed means; full table in Table 2). Both internal strategies flatten as the budget grows: reasoning-only at its information ceiling, and best-of-$N$ at 86.7% *even with an oracle (ground-truth-test) verifier*, capped by the high-probability part of the model's own output distribution. The two external strategies import execution feedback each cycle and climb past that ceiling, the proposer–reviewer harness to a strict 100% with zero seed variance. We run this experiment in Section 4.

was never going to write. Longer thinking rearranges the known; more samples explore the already-likely. We call the resulting plateau the **internal ceiling** (formalized in Appendix A via the data-processing inequality; since that bounds an information channel rather than achievable quality, our evidence is experimental). Figure 5 makes it concrete: reasoning-only and *oracle-verified* best-of-$N$ flatten as the budget grows, while interaction strategies climb past them to a perfect pass rate. Grounded feedback escapes the ceiling because the observation is computed from the artifact's real behavior (a failing assertion, a measured bounding box, a search verdict); conditioning the next proposal on it lets the model synthesize a candidate it would never have sampled unaided. That is what separates interaction from selection (which cannot create a missing candidate) and from reasoning (which adds no information): a new information channel, not a constant-factor speedup.

**The symmetric claim: an ungrounded metric cannot measure the gain.** The same argument applies to the *scorer*: a model judging a lossy view of the artifact (a VLM reading a cropped screenshot) cannot certify an improvement that lives in the part its channel drops. Grounding the evaluation is therefore a precondition for *detecting* interaction scaling wherever quality is not fully visible to the default judge, exactly the situation for rendered visual artifacts (Section 6).

### 2.4 Three testable predictions

**Prediction 1.** At a matched token budget, reasoning-only and best-of-$N$ scaling saturate strictly below what interaction reaches (Section 4). **Prediction 2.** Holding the task suite fixed and swapping only the reviewer's feedback channel, improvement tracks the instrument's *coverage* of the defects present, not the act of reviewing (Section 5). **Prediction 3.** On modalities whose quality is invisible to the default judge, the gain is only detectable with a grounded metric, and

**Table 1.** Modality instantiations. Each modality pairs a grounded feedback channel (named per the taxonomy of Figure 4) with the instrument that both drives revision and scores the result. "†" marks the four modalities scored by the deterministic DOM-geometry instrument and analyzed jointly in Section 6.

| Modality | Feedback channel | Grounded instrument | Hard suite ($N$) |
|---|---|---|---|
| Code | execution | `pytest` pass/fail + traceback | 15 (+11 deep-spec) |
| Academic figures† | geometry | DOM geometry + alignment | 20 |
| Slides† | geometry | DOM geometry + alignment | 12 real-paper |
| Web pages† | geometry | DOM geometry, multi-width | 20 |
| Animations† | temporal | DOM geometry, per-frame | 20 SVG/CSS |
| Video editing | exec. + temporal | script exec + Gemini-native rubric | 15 |
| Deep research | external facts | search-grounded factual rubric | 15 |

an ungrounded reviewer can even regress quality (Sections 5 and 6).

## 3 Setup: A Budget-Aware Proposer–Reviewer Harness

**Architecture.** The harness is pure scaffolding around a *frozen* frontier model (Claude Sonnet 4, `claude-sonnet-4-20250514`, temperature 0 unless noted): no fine-tuning, no retrieval, no learned controller. A **proposer** produces an artifact. An **instrument** executes, renders, or measures it to produce a grounded signal. A **reviewer** (the same model in a diagnostic role) turns that signal into a structured list of defects, and the proposer revises. The loop repeats up to an iteration cap and *keeps the best-scoring iteration*, so the reviewed result can never score below single-shot under the same metric. Single-shot is just one proposer call.

**The three grounded instruments.** Every modality is paired with a deterministic or near-deterministic instrument that both *feeds back* and *scores*:

- **Execution** (`pytest`): exact pass/fail plus the failing assertion and traceback. Each task ships a multi-assertion suite targeting the edge cases that separate correct from plausible-but-wrong (e.g. the semantic-version comparator is checked on pre-release precedence), and the deep-spec suite is additionally validated against a reference implementation. A "pass" therefore certifies the specified behavior, and the oracle best-of-$N$ baseline of Section 4 selects against these *same* tests, so its ceiling is measured by the identical criterion.
- **Rendered geometry**: the artifact is rendered headless and every element's true bounding box is read from the layout engine. We compute, exactly: text-on-text overlap ($\geq 6$ px), out-of-bounds/clipping, container overflow, document overflow ($> 16$ px), and *box-group misalignment* (rows or columns of card-like boxes flagged for unequal size, misaligned far edges, or uneven gutters). Web pages are scored at desktop and mobile widths; animations are probed at each sampled frame.
- **Native video** (Gemini 3.1 Pro): the rendered clip is judged whole against per-requirement binary checks, a near-grounded substitute where pixels, not a still, are observed.

Where no deterministic instrument exists (flowing web content, factual research), we fall back to a binary per-requirement rubric, flagged as such.

All artifacts are produced under a common *design-principle* generation prompt (proximity,

alignment, repetition, contrast, deliberate color) so that single-shot quality is already strong: the headroom we measure reflects genuine task difficulty, not a weak prompt.

**Budget and protocol.** A run has a token budget $B$ split across proposer and reviewer calls; a sweep of the split (Table 9) shows pass rate rises monotonically with the proposer's share, so we allocate the majority to generation, and Table 1 lists the per-modality instruments. Unless noted, the proposer runs at temperature 0, and each modality is evaluated over three independent seeds, keeping the best-scoring iteration (cap $\leq 3$ for geometry, $\leq 5$ for code); we compare single-shot against reviewed on each (task, seed) pair. For each result we report a two-sided paired sign test over the decisive pairs and, for the geometry and code effects, a paired-bootstrap 95% confidence interval on the effect size; exact values accompany each figure.

## 4 Interaction Scaling at a Matched Token Budget

**The saturation experiment (Prediction 1).** On the hard code suite, at a fixed per-task token budget, we compare four strategies: reasoning-only (extended thinking), best-of-$N$ sampling with an *oracle* verifier, a single-agent loop, and the proposer–reviewer harness. The result is Figure 5, previewed in Section 2.3: both internal strategies flatten as the budget grows, while *every* strategy that iterates on feedback climbs past them toward a perfect pass rate. The best-of-$N$ comparison is the sharpest, because its verifier is the ground-truth test suite itself, so its ceiling is effectively pass@$N$: the tasks it never solves are those where none of its samples passes, and a perfect selector cannot return a candidate the proposer never writes. The harness solves those same tasks by conditioning the next draw on execution feedback. The gap is information-theoretic, not a tuning artifact.

**Code as the clean case study.** Code is the cleanest demonstration, because the instrument is exact and the score is objective. The harness recovers every first-shot failure with no regressions, on both the development suite and a harder deep-spec suite of from-scratch implementations (a JSON parser, a spreadsheet-reference resolver, a minimal edit-script diff, an expression evaluator), lifting both to a perfect 100% pass rate (Figure 6). The mechanism is the same across recovered traces: turn 1 produces code that compiles but mishandles an edge case (an off-by-one boundary, semantic-version comparison, CIDR arithmetic); turn 2 receives the failing assertion, the offending input, and the expected-vs-actual values, and rewrites just that block; turn 3 confirms the fix. Reasoning alone cannot supply this: the model cannot know its boundary condition is wrong until a concrete test exercises it.

**Architecture buys efficiency and reliability, not ceiling.** Among interaction strategies, the ceiling pass rate ties within seed noise; what separates them is cost and variance (Figure 7). The proposer–reviewer harness is the most token-efficient, and the only variant that reaches 100% in *every* seed. Three things explain the efficiency: the reviewer sees only what it needs to locate defects, it returns a structured list rather than raw `stderr`, and the two roles are specialized.

**Built-in early stopping.** The loop does not over-revise: on code, *every* already-passing single-shot task is submitted at iteration 1 with no revision and no wasted tokens. The grounded

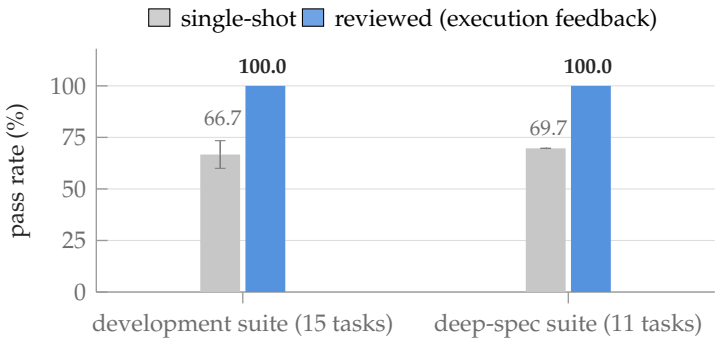



**Figure 6. Execution feedback recovers every first-shot failure on both code suites.** Single-shot vs. harness-reviewed pass rate (development suite: 3-seed mean, whisker ±1 SD; deep-spec: sign test $p = 0.002$). Every failing run is recovered and no passing run regresses; the reviewed bars carry zero seed variance.

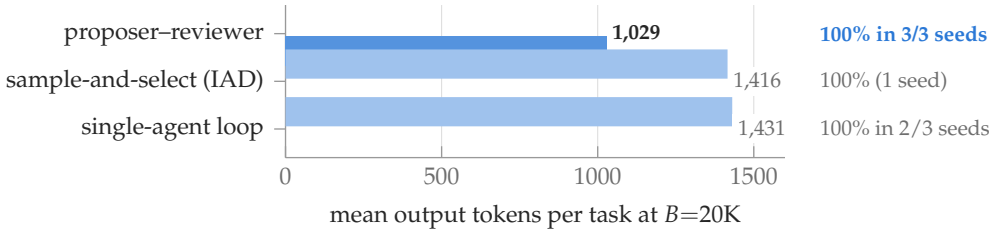



**Figure 7. Same ceiling, different cost and reliability.** All three interaction strategies use execution feedback and tie on pass rate within seed noise (sign test $p > 0.6$); the proposer–reviewer harness converges with ∼28% fewer tokens and is the only variant at 100% across all seeds (Table 5).

signal (a failing test) is the trigger; without it the loop is silent. This is why the harness's token overhead over single-shot stays modest despite a generous iteration cap.

**The effect is architectural, not Claude-specific.** Over three seeds per family, the harness lifts all three we test: Sonnet 4, Qwen3-235B, and GPT-5 (the last from a higher single-shot baseline; Figure 8). The telling detail is the variance: single-shot fluctuates seed-to-seed for every family, while the reviewed ceiling has *zero* seed variance for all three, converging to the same point regardless of the starting draw. The result also holds out of sample: on a 32-task held-out suite built after the method was fixed, the harness again recovers every first-shot failure with no regressions and reaches a perfect pass rate (Table 8; the absolute lift is smaller only because the held-out single-shot baseline is higher).

## 5 Feedback-Side Grounding: Swapping the Reviewer's Instrument

The saturation experiment compared strategies. This section holds everything else fixed (task suite, proposer model, number of reviewing passes) and swaps *only the reviewer's feedback channel*, testing the coverage principle (Prediction 2) directly on both a textual and a visual modality.

**On code: execution feedback wins where its coverage lies.** We run three configurations on the fixed code suite, each with *exactly one* reviewing pass: ungrounded critique (the LLM

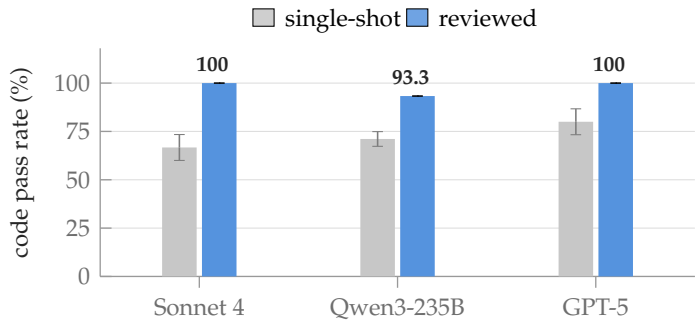



**Figure 8. The execution loop replicates across model families.** Single-shot vs. harness-reviewed code pass rate for three families, three seeds each (lifts +33.3/+22.2/+20.0 pp; whiskers ±1 SD across seeds). The reviewed bars carry *zero* seed variance for all three families.


**Figure 9. Swapping only the feedback signal on a fixed code suite: improvement tracks coverage, and cost tracks it decisively.** One reviewing pass per configuration. Ungrounded critique (orange), critique + linter (light blue: grounded, but observing only *form*), and critique + test execution (blue: grounded, observing *behavior*). The linter configuration matches bare critique on these runtime-logic bugs (exactly what the coverage principle predicts) while the execution configuration reaches a higher ceiling at ∼2.5× lower cost and fewer iterations (1.53 vs. 2.07; per-seed detail in Table 4).

re-reads the code), static form (the same critique plus `ruff` linter output), and execution (the same critique plus the failing test's traceback). Any difference between them therefore isolates the *signal*, not the act of reviewing. Figure 9 shows both outcomes. The gap in pass rate is modest and rests on a single leaner-budget seed, so we read it cautiously. The clear gap is in *cost*: the execution configuration converges roughly 2.5× cheaper and in fewer iterations, because a concrete failing assertion pinpoints the fix where an ungrounded critique can only guess. The static-form configuration is the coverage principle's cleanest test: the linter is genuinely grounded, but the seeded bugs are runtime-logic errors that leave no trace in the source, so it buys nothing over bare critique. Grounding helps only as far as the instrument can see.[1]

**On slides and figures: an uncovered instrument makes things worse.** The decisive version of this control is visual. We fix the task suite and the proposer, and give the reviewer either (i) the deterministic geometry report (measured boxes, exact overlaps) or (ii) a VLM's reading of

[1]Symmetrically, the principle predicts a static-form lift on a bug set that *is* visible in form (type confusions catchable by `mypy`, taint patterns catchable by `semgrep`). Constructing that suite is future work; on our behavioral bug set the null result is the prediction.

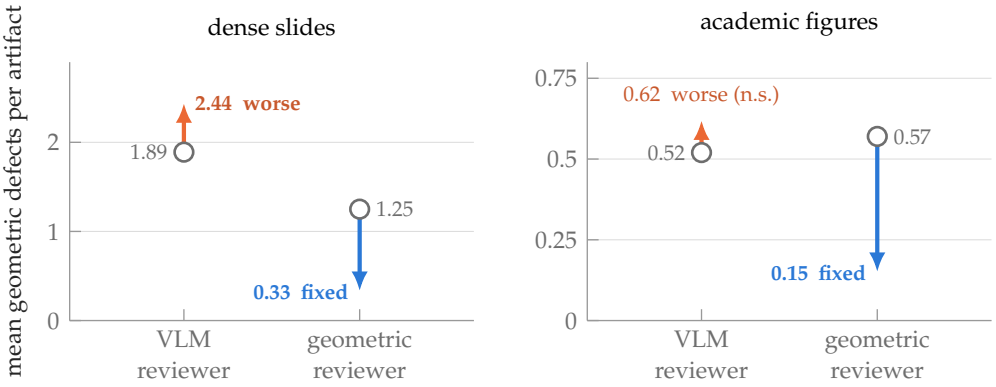



**Figure 10. Same tasks, same proposer, one reviewing pass: the arrow's direction is set by the reviewer's instrument.** Each arrow runs from that configuration's own single-shot baseline (open circle; the configurations are separate runs, hence differing baselines) to its reviewed result, in mean geometric defects per artifact (lower is better; note the differing $y$ scales). The screenshot-fed VLM reviewer moves *up* on slides and directionally up on figures; the deterministic geometric reviewer moves sharply *down* on both (slides $-73\%$, $p = 0.0018$; figures $-74\%$, $p = 7\times10^{-4}$).

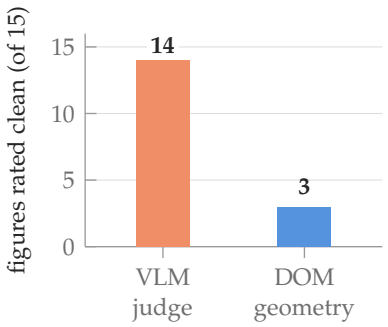
a screenshot, which crops the very defects at issue. The two configurations are separate runs, so each is compared against its *own* single-shot baseline; what matters is the *direction* each reviewer moves that baseline (Figure 10). The geometry reviewer sharply reduces real layout defects on both modalities. The VLM reviewer *increases* defects on slides and is directionally worse on figures: blind to the true geometry, its edits break alignment about as often as they fix it. One reviewing pass, opposite sign, decided entirely by whether the signal covers the defect. (On the same slides, the binary VLM *rubric* saturates at the same time; Section 6 takes up that evaluation-side half.)

**What this establishes.** On the feedback side, grounding is necessary, but the quantity that decides the outcome is *coverage*. A grounded instrument that cannot see the defect (the linter, on runtime bugs) matches ungrounded critique, and an instrument that actively drops the defect (the screenshot) is worse than no reviewer at all. The remaining question is symmetric: what happens when the *metric* is such an instrument?

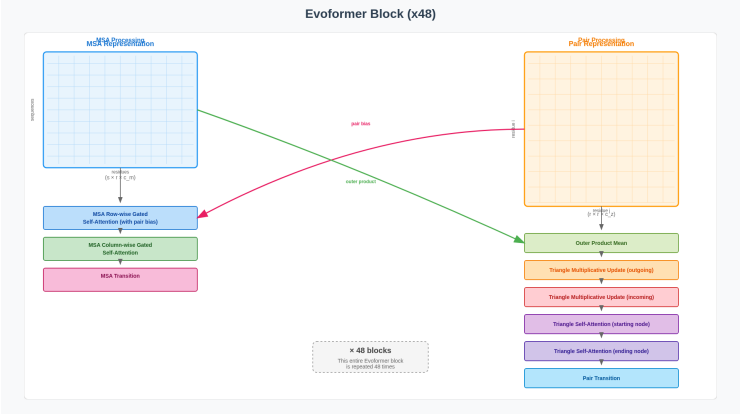
## 6 Evaluation-Side Grounding: Deterministic Instruments vs. Model Judges

The visual modalities expose the symmetric half of the framework (Prediction 3): even when grounded feedback *does* fix the artifact, an ungrounded metric cannot see the fix.

**The default VLM judge is structurally blind.** The field's default evaluator for visual artifacts is a binary per-requirement rubric scored by a VLM from a screenshot. On an initial probe of 15 dense academic-paper architecture figures (even with native-resolution quadrant tiling), this judge rates 14 of 15 single-shot renders "perfect," meaning they satisfy every *content* requirement it checks. Yet by direct inspection most of those renders are broken, with section titles printed on top of each other and labels spilling out of boxes (Figure 11b). The failure is mechanical, and no better prompt fixes it: screenshots are captured at a fixed resolution, so content overflowing the canvas is cropped *off-frame before the image reaches the judge*, and




**(a)** The VLM passes 14/15; the deterministic instrument, 3/15.




**(b)** A single-shot figure the VLM rated "perfect": both section titles are superimposed ("MSA Processing" over "MSA Representation"; "Pair Processing" over "Pair Representation") and axis labels collide with the matrix borders.

**Figure 11. The default VLM judge is structurally blind to layout defects. (a)** On the same 15 single-shot academic figures, the VLM-on-a-screenshot judge rates 14 "perfect" while the DOM-geometry instrument finds only 3 actually clean; the other 11 are broken in ways cropped off-frame or below VLM acuity. **(b)** A representative "perfect"-rated render whose text-on-text overlaps are exactly what the geometry check flags and the screenshot judge misses.

small in-frame overlaps fall below its acuity. The DOM-geometry instrument, reading the same artifacts' actual bounding boxes, finds only 3 of 15 truly clean (Figure 11a). This is not a wrong answer to the same question; the judge is *structurally unable to observe* the property that matters, an instrument whose coverage excludes the defect, used as the score. (This probe is a deliberately hard, unconstrained set; the suites scored below use the design-principle prompt of Section 3 and are correspondingly cleaner single-shot.)

**A grounded instrument reveals a large, decisive effect on all four modalities.** We score four visual modalities with the same DOM-geometry-plus-alignment instrument under one identical configuration (Sonnet 4, temperature 0, design-principle prompt, three seeds, propose $\rightarrow$ measure $\rightarrow$ feed exact defects back $\rightarrow$ revise, $\leq 3$ iterations). Figure 12 summarizes and Table 6 gives the full statistics: grounded geometry feedback removes a large fraction of the real layout defects on *all four* modalities, with every bootstrap confidence interval excluding zero and improvements outnumbering regressions by roughly ten to one. (Figure 3 in the introduction shows the per-artifact reality behind these aggregates.) The size of the effect tracks single-shot headroom: dense responsive web pages are full of defects out of the box, while a frontier model under a design-principle prompt already lays out many figures and slides cleanly. Animations carry the widest interval, because their per-task defect counts are heavy-tailed; there we treat the sign test as the robust statement and the mean as indicative.

**Is the reduction circular?** The instrument both *drives* the revision and *scores* it, and the harness keeps the best-scoring iteration, so *some* reduction is mechanically guaranteed. The real questions are how large it is, and whether it reflects quality a human would care about. Three facts argue it is not just an artifact of optimizing the reported number. **(i) The defects are real:** single-shot renders carry them at high rate (Figure 11b), and the thresholds are

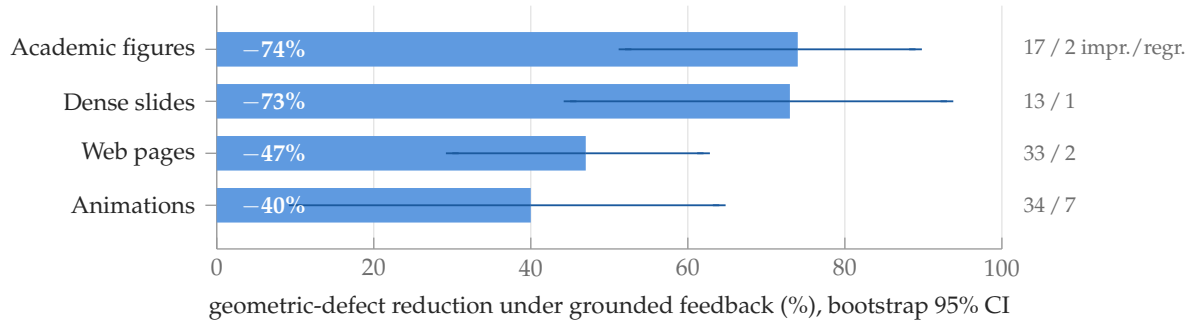



**Figure 12. Grounded geometric feedback removes real layout defects on all four visual modalities.** Mean defect reduction under one identical configuration (3 seeds; whiskers are paired-bootstrap 95% CIs, all excluding zero; right column counts improved vs. regressed task-runs; all paired sign tests $p < 2\times10^{-3}$; full statistics in Table 6). The standard VLM-on-a-screenshot metric measures none of this.

conservative, so a flagged defect is visible, not sub-perceptual. **(ii) The magnitude is not preordained:** keeping the best of three iterations could move the metric by almost nothing; that grounded revision instead removes most of the real defects (Figure 12) is a property of the feedback, not of the keep-best rule. **(iii) The ungrounded-reviewer ablation is the decisive control:** scored by the *same* metric, the VLM-feedback configuration fails to reduce it and worsens slides (Figure 10). So the reduction tracks the grounding of the feedback, not the act of optimizing the scorer, a conclusion the model-free cross-model replication below reinforces. The principal remaining caveat is a quantitative human-preference study confirming that DOM-defect reduction maps onto perceived quality across the full suite.

**The geometry effect is not proposer-specific.** To rule out a Claude- or prompt-specific explanation, we run the same geometry-feedback harness with **Gemini 3.1 Pro** as proposer on the dense real-paper slide suite, scored by the identical model-free DOM instrument. The effect reproduces and is larger: Gemini's single-shot slides carry more defects than Sonnet's, and the grounded loop removes almost all of the excess (−93% vs. −73% for Sonnet, 19 of 20 decisive task-runs improved). Since the instrument uses no model, the effect is neither proposer- nor scorer-specific.

**Two modalities where the honest answer is "no headroom."** *Video editing*, scored by Gemini 3.1 Pro's native full-clip rubric (near-grounded, since it observes the pixels, not a still), is already strong single-shot, so the reviewed lift is small and not significant; a hardened multi-step suite de-saturates it. *Deep research*, scored against exact provided facts, saturates: a frontier model knows well-documented facts and does not fall for the planted traps, so the factual-feedback lift is capped by a near-zero single-shot error rate. This is a genuine limit of the modality, since de-saturating it would require facts the judge itself cannot grade. We report both as scoped negatives, not headline lifts (Figure 2).

## 7 Internalizing the Harness into a Small Student

Two things could carry the harness's gains: the grounded scaffolding around the model, or behavior absorbed into the weights. This section asks how much is the latter: whether the loop's interaction quality can be *internalized*, so a small model produces high-quality artifacts

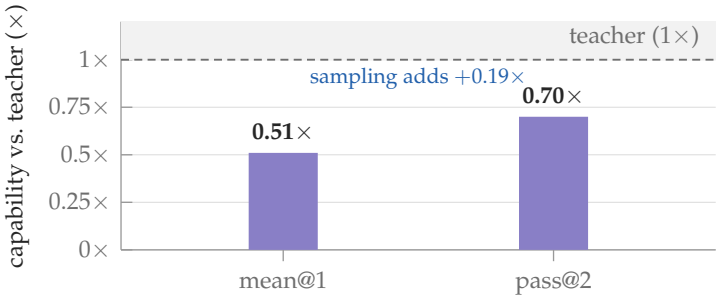



**Figure 13. An 8B student internalizes much of the teacher's interaction quality.** Distilled student as a fraction of the frontier teacher on the 44-task out-of-distribution suite (Table 12): one sample recovers 0.51× and two samples 0.70× of teacher capability, at ∼10× lower deployment cost. On the 18-task hard held-out suite the same student passes 44% at pass@1 and 56% at pass@3 (Table 11).

in fewer passes. This complements the thesis rather than replacing it, because the cheaper internalized proposer still runs *inside* the same grounded harness. We distill judge-filtered teacher trajectories from the harness into an 8B Qwen3-VL student by supervised fine-tuning.

**Result.** On an out-of-distribution suite, the student recovers about half the teacher's single-sample capability, and a second sample closes much of the remaining gap (Figure 13), at roughly 10× lower deployment cost. On a harder held-out suite of the teacher's own pre-curated failures it passes a substantial fraction (Table 11), and run back inside the harness it recovers still more.

**Variance is a budget (a cautionary finding).** Reinforcement fine-tuning (RFT) on top of SFT improves every per-turn consistency metric we measure, *and* it lowers pass@$k$ (Figure 14). The two students start nearly tied on a single sample; as samples are added, the SFT student keeps turning fresh draws into newly solved tasks, while the RFT student's curve goes flat. When you deploy by sampling, the variance in the output distribution *is* the lever that inference scaling pulls, so post-training that reduces that variance spends from the very budget best-of-$N$ relies on. Variance here is not noise to minimize; it is the resource sampling converts into quality. The recipe, then: use SFT to internalize interaction quality, keep the sampling temperature, and run the student inside the grounded harness.

## 8 Related Work

**Test-time compute scaling.** Two axes are established. *Reasoning* scaling (chain-of-thought, tree search, and extended thinking (Wei et al., 2022; Yao et al., 2023a; DeepSeek-AI, 2025; Snell et al., 2025; Team, 2025)) spends more tokens before committing. *Sampling* scaling (self-consistency and best-of-$N$ (Wang et al., 2023b; Brown et al., 2024)) draws multiple attempts and selects one. Both only rework the same frozen weights and prompt (both are *internal* in the sense of Section 2, which makes precise how they are bounded). We position *interaction* as a third, external axis that imports information the model does not have.

**Self-correction and critique.** Iterative refinement with model-generated feedback (Reflexion (Shinn et al., 2023), ReAct (Yao et al., 2023b), Self-Refine (Madaan et al., 2023), CRITIC (Gou

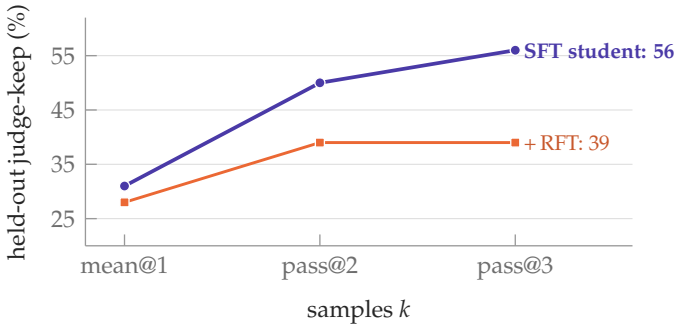



**Figure 14. RFT polishes consistency but spends the variance that sampling needs.** Judge-keep vs. number of samples on the 18-task hard held-out suite for the chosen SFT student and the same student after RFT. RFT improves every per-turn consistency metric (Table 15) yet regresses pass@3 by 17 pp: the near-tie at one sample and the flat curve past it show the lost quality is exactly the distributional spread best-of-$N$ converts into solved tasks.

et al., 2024)) is widely used, but its reliability is contested: Huang et al. (2024) show that models often cannot self-correct reasoning without an external signal, and Kamoi et al. (2025) chart when correction does and does not help. Our taxonomy makes the distinction explicit: *ungrounded* critique is subject to the internal ceiling, while refinement against an executed or measured observation is not. Our controls (Section 5) then show that it is the instrument's coverage of the defect, not the extra critique pass, that carries the gain.

**Grounded and execution feedback.** Tool use and execution feedback (Toolformer (Schick et al., 2023), Gorilla (Patil et al., 2024), Voyager (Wang et al., 2023a), RLEF (Gehring et al., 2025), and, for website generation specifically, WebGen-Agent's multi-level visual/functional feedback (Lu et al., 2025)), together with the thinking-vs-doing analysis of Shen et al. (2025), establish that acting on an environment helps. We contribute the information account of *why*, a coverage principle that predicts *when*, and, crucially, the requirement that the *evaluation* be grounded too. We also compare harness architectures (single-agent loops (Tran and Kiela, 2026), sample-and-select (Ruan et al., 2025), proposer–reviewer) under a matched budget.

**Model-as-judge reliability.** LLM- and VLM-as-judge evaluation is now standard (Zheng et al., 2023), and its failure modes (position, verbosity, and self-preference biases, and broader reward-model fragility (Casper et al., 2023)) are documented. Our finding is sharper and, to our knowledge, not previously isolated: for artifacts whose quality is a measurable *physical* property (the geometry of a rendered layout), a screenshot judge is not merely biased but *structurally blind* (the defect is dropped by its observation channel before judgment begins), so it both fails to score the defect and, used as a reviewer, fails to fix it. This is why a deterministic instrument is necessary, not merely preferable, to detect interaction scaling on visual modalities.

**Practitioner evidence at scale.** Industrial deployment corroborates the gap our framework targets. Hong (2026) reports a controlled study at ByteDance (3 frontier coding models $\times$ 3 agent frameworks $\times$ 100 runs) on a single real product feature: functional *correctness* exceeded 80% for every pair, yet "deliverability" axes (usability, reliability, maintainability, performance) collapsed to 40–60% and rose to $\sim$80% only after harness "infrastructure" was added, while

correctness barely moved. This is the signature we formalize: internal scaling saturates the easy-to-see axis while the interaction loop carries the rest. Tellingly, the report had to define a multi-axis quality metric before the gap was visible at all (an industrial analogue of our grounded-evaluation requirement), and its grounded fixes (e.g. browser-driven validation before commit) are instrument observations in our taxonomy, not ungrounded self-review.

**Information theory.** The data-processing inequality (Cover and Thomas, 2006) underlies the formal version of the internal ceiling (Appendix A): any post-processing of the model's outputs (reasoning, ungrounded review) cannot increase information about the target, whereas an instrument observation introduces a new channel.

## 9 Limitations

Two modalities saturate: video editing is already strong single-shot (the lift is real only on a hardened multi-step suite), and deep research saturates because frontier models know well-documented facts and the judge cannot grade facts it does not have. Deterministic geometry applies only to artifacts with a measurable intended layout; for genuinely flowing content we still rely on a binary rubric, with its VLM-reliability caveat. Our static-form tier is confirmed only on its null prediction (no lift on bugs invisible in form), not yet on a positive one. And because the geometric instrument both supplies the feedback and scores the result, the reported reductions are partly an optimization of the metric itself; we argue in Section 6 (conservative thresholds, non-trivial magnitude, and the ungrounded-reviewer control that moves the same metric the wrong way) that they reflect real, human-visible defects, but a quantitative human-preference validation remains future work.

## 10 Conclusion

Grounding is the load-bearing variable, and it must hold on both sides of the loop: the feedback that drives revision and the metric that scores it. Reasoning and sampling are internal and hit an information ceiling; interaction escapes it because a grounded instrument imports a real observation each cycle, bounded only by what that instrument can observe. The practical lesson is not "add more loops" (swyx and Latent Space, 2026; Runkle, 2026) but "ground the loop you add": wrap a frozen model in a proposer–reviewer harness whose instrument covers the defects that matter, and for visual artifacts measure the rendered DOM, never a screenshot, on both the reviewer and the scorer. Much recent progress on self-improving visual generation is measured with VLM judges that cannot see the defects at issue; grounding the metric is a cheap, deployable correction. Two directions follow: deterministic instruments for modalities beyond layout and execution (audio, 3D, tabular, UI-interaction artifacts), where model judges are likewise blind, and tightening the internal ceiling from an information bound into a quantitative relationship between coverage and achievable quality. Interaction scaling is real, distinct from reasoning and sampling, and predictable from coverage; once you ground the metric, it is plainly visible.

## Acknowledgements

This paper was produced using Pine Copilot's voice-directed *whisper coding* workflow (Pine AI, 2026), in which the authors specify, discuss, and review the work by voice while a coding agent (Claude Code with Claude Opus 4.8) carries out the planning, coding, experiments, and paper writing. We thank BSQL Networking for hosting the NVIDIA RTX PRO 6000 GPU.

## References

Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling, 2024.

Stephen Casper, Xander Davies, Claudia Shi, Thomas Krendl Gilbert, et al. Open problems and fundamental limitations of reinforcement learning from human feedback, 2023.

Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning, 2025.

Jonas Gehring, Kunhao Zheng, Jade Copet, Vegard Mella, Taco Cohen, and Gabriel Synnaeve. RLEF: Grounding code LLMs in execution feedback with reinforcement learning. In *International Conference on Machine Learning*, 2025.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations*, 2024.

Dingkun Hong. The practice and exploration of AI coding. Keynote, ByteDance/Volcano Engine Force Conference, Beijing; speaker is VP of Engineering at ByteDance, 2026. https://mp.weixin.qq.com/s/tJAinVKzZAqZrhiEvGU2Dg.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations*, 2024.

Ryo Kamoi, Yusen Zhang, Nan Zhang, Jiawei Han, and Rui Zhang. When can LLMs actually correct their own mistakes? a critical survey of self-correction of LLMs. *Transactions of the Association for Computational Linguistics*, 2025.

Zimu Lu, Houxing Ren, Yunqiao Yang, Ke Wang, Zhuofan Zong, Junting Pan, Mingjie Zhan, and Hongsheng Li. WebGen-Agent: Enhancing interactive website generation with multi-level feedback and step-level reinforcement learning, 2025.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Sean Welleck, Bodhisattwa Prasad Majumder, Shashank Gupta, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023.

OpenAI. OpenAI o1 system card, 2024.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems*, 2024.

Pine AI. Pine AI: The most natural human-computer interface is your voice. Blog post, 2026. URL https://www.19pine.ai/blog/pine-ai-the-most-natural-human-computer-interface-is-your-voice. Accessed 2026-06-28.

Yangjun Ruan, Eleftheria Briakou, Charles Xie Han, Yu Chen, Yufan Jiao, et al. Iterative agent decoding for detecting compounding errors in LLM agents, 2025.

Sydney Runkle. The art of loop engineering. LangChain (blog), 2026. https://www.langchain.com/blog/the-art-of-loop-engineering.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessí, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.

Junhong Shen, Hadi Jain, Zelai Xiao, Brandon Amos, Aaditya Ramdas, Yuandong Tian, and Alborz Geramifard. Thinking vs. doing: Agents that reason by scaling test-time interaction. In *Advances in Neural Information Processing Systems*, 2025.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. In *International Conference on Learning Representations*, 2025.

swyx and Latent Space. Loopcraft: The art of stacking loops. Latent Space (blog), 2026. https://www.latent.space/p/ainews-loopcraft-the-art-of-stacking.

Kimi Team. Kimi k1.5: Scaling reinforcement learning with LLMs, 2025.

Dat Tran and Douwe Kiela. Single-agent LLMs outperform multi-agent systems on multi-hop reasoning under equal thinking token budgets, 2026.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. In *NeurIPS Foundation Models for Decision Making Workshop*, 2023a.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023b.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, 2023a.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023b.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

## A Formal Statement of the Internal Ceiling

This appendix formalizes the one-sentence argument of Section 2.3. Let $A^\star$ be the (task-determined) correct artifact and $A$ a candidate. The proposer is a channel from its inputs $\Theta$ (weights) and prompt $X$ to $A$. A *reviewer* that only re-reads $A$ and reasons (ungrounded critique) forms the Markov chain $A^\star \to (\Theta, X) \to A \to R$, where $R$ is its critique. By the data-processing inequality (Cover and Thomas, 2006),

$$I(A^\star; R) \;\le\; I(A^\star; (\Theta, X)),$$

so no amount of reasoning or ungrounded review can recover more information about $A^\star$ than is already in $(\Theta, X)$. This bounds reasoning-only scaling. Sampling is bounded differently, but no less tightly: for any finite $N$, best-of-$N$ can only *select* among the $N$ candidates the proposer actually draws, so even an oracle (grounded) verifier cannot return a correct artifact the proposer is too unlikely to sample within the budget. Its practical ceiling is therefore the high-probability part of the proposer's own output distribution, not a new information channel.

Grounded feedback escapes both ceilings by feeding an instrument observation $E = g(A, \text{world})$ (a test outcome, a measured bounding box, a search verdict) back into *generation*. Conditioning the next proposal on $E$ lets the proposer synthesize a candidate it would not otherwise have sampled: formally, $I(A^\star; (R, E))$ can exceed $I(A^\star; (\Theta, X))$, because $E$ carries information obtained by running the artifact against reality (the tests encode the specification; the layout engine encodes real rendering semantics the model only approximates). The *coverage* qualification of Section 2.2 enters here. $E$ helps only to the extent that $g$'s observation is informative about the defects actually present; a linter's $g$ observes form, so on runtime-logic bugs $I(A^\star; E)$ adds nothing actionable, which is what Figure 9 measures.

The symmetric claim covers the scorer. A metric $M$ that is itself a model's function of a lossy view of $A$ (a VLM judging a cropped screenshot) cannot certify an improvement that lives in the part of $A$ its channel drops. Grounding the evaluation is therefore a precondition for *detecting* interaction scaling on any modality whose quality is not fully visible to the default judge.

Two cautions. The inequality bounds the *information channel*, not achievable quality directly; we use it to motivate the saturation that Figure 5 then measures, and treat the measurement as

the evidence. And grounded interaction is of course still bounded, by what the instrument exposes and by the proposer's ability to act on it.

## B Detailed Results and Configurations

This appendix collects the full tables underlying the figures in the main text, from the four-strategy scaling curves through the distillation ablations.

### B.1 Four-strategy scaling at a matched 20K-token budget

**Table 2. The headline scaling result (3-seed mean ± SD), underlying Figure 5.** Pass rate by strategy × budget on 15 hard code tasks, Sonnet 4. S, L, and H are averaged over three independent seeds; R was run once (its seed-to-seed variation is small at temp 0), at the sweep's budget partition (a more generous partition lifts R to 86.7%; see Table 3). S's small dip from $B$=1K (80.0%) to $B$=5K (77.8%) is within the ±3.1 pp seed noise: pass@$N$ is non-decreasing in $N$ in expectation, but not on every finite sample. Reasoning-only and best-of-$N$ both saturate well below 100%, while all three feedback-iterating strategies reach ≥ 97.8% at $B$=20K (L hits 100% on 2 of 3 seeds, H on all 3, so H is the only variant at a strict 100% across seeds). H and L tie on pass rate within seed noise; what separates them is token efficiency and reliability (Figure 7).

| Budget $B$ | R (reasoning-only) | S (best-of-$N$) | L (single-agent loop) | H (proposer–reviewer) |
|---|---|---|---|---|
| 1K | 60.0% | 80.0% ±0.0 pp | 57.8% ±3.1 pp | 62.2% ±3.1 pp |
| 5K | 73.3% | 77.8% ±3.1 pp | **93.3% ±0.0 pp** | 91.1% ±3.1 pp |
| 20K | 73.3% | 86.7% ±0.0 pp | 97.8% ±3.1 pp | **100.0% ±0.0 pp** |
| Ceiling | 73.3% | 86.7% | ∼100% | **100% (zero variance)** |

### B.2 Reasoning-only at a matched budget

**Table 3.** Reasoning-only at a matched budget vs. single-shot and the harness on the 15 hard code tasks. These are single-seed numbers (the reference seed, also used in Table 4); the 3-run aggregate is in Table 8 (66.7 ± 6.7% single-shot, 100.0 ± 0.0% reviewed). *Reading the numbers:* single-shot on this seed is 11/15 = 73.3% (the canonical 3-seed mean is 66.7%), and the reasoning value shown is the *thinking-heavy* partition (86.7%). Two numeric coincidences of this small suite can mislead: 86.7% here matches the oracle best-of-$N$ ceiling, and the *default*-partition reasoning ceiling (73.3%, Table 2) matches this seed's single-shot; neither is the same run. Reasoning closes two-thirds of the harness's gain over single-shot, but stays 6.7 pp short of the harness even at 1.57× its token budget.

| Strategy | Pass rate | Mean tokens | Δ vs. single-shot |
|---|---|---|---|
| Single-shot | 73.3% (11/15) | 1,406 | n/a |
| Reasoning-only (thinking-heavy partition) | 86.7% (13/15) | 4,137 | +13.3 pp |
| **Proposer–reviewer harness** | **93.3% (14/15)** | **2,643** | **+20.0** pp |

### B.3 Feedback-channel control on a fixed code suite

**Table 4.** Feedback-channel controls on 15 hard code tasks (Sonnet 4 proposer and reviewer), underlying Figure 9. Each row is a single seed. "SS" is that seed's single-shot pass rate (it varies by row), and "Reviewed" is the ceiling after that seed's harness run. The three SS values (10/15 and 11/15) are draws from the same single-shot distribution behind the 3-run mean of 66.7 ± 6.7% (Table 8; within ±1 SD of 10/15). Since SS varies by row, the comparison that matters is the *reviewed* ceiling: execution reaches 14/15, critique and critique+linter reach 13/15 (+6.7 pp), *and* execution is ∼2.5× more token-efficient (2,643 vs. ∼7,100 tokens/task, 1.53 vs. 2.07 iterations). The one task no control row recovers is `code_011` (CJK text wrapping); the full harness at $B$=20K recovers it on all three seeds (Table 2), which is why the headline reviewed rate is a strict 100% while this leaner single-seed control caps at 14/15.

| Feedback channel | Reviewer sees | SS | Reviewed | Iters |
|---|---|---|---|---|
| None (single-shot) | n/a | 11/15 | n/a | n/a |
| Ungrounded critique | code only | 10/15 | 13/15 | 2.07 |
| + linter (static form) | code + `ruff` output | 11/15 | 13/15 | 2.07 |
| + **execution** | code + test stderr/traceback | 11/15 | **14/15** | **1.53** |

### B.4 Single-agent loop vs. proposer–reviewer harness

**Table 5.** Three interaction-strategy variants at $B$=20K on the 15 hard code tasks, all using execution feedback; underlying Figure 7. Pass rate is within seed noise (sign-test $p$>0.6 on the H–L comparison). The harness wins on *token efficiency* (1,029 vs. 1,431 L vs. 1,416 IAD mean output tokens) and on *seed variance* (zero across three seeds for H, while L hit 100% in 2 of 3 seeds and IAD was run for one seed only).

| Strategy | Pass rate | Mean tokens | Notes |
|---|---|---|---|
| L (single-agent loop) | 97.8% ±3.1 pp (3 seeds) | 1,431 | 1 task in 1 seed |
| IAD (oracle, K=3, T=0.7) | 100.0% (1 seed) | 1,416 | +38% tokens vs. H |
| **H (proposer–reviewer)** | **100.0% ±0.0 pp (3 seeds)** | **1,029** | zero seed variance |

### B.5 Deterministic geometric-feedback statistics

**Table 6.** Full statistics for Figure 12: deterministic geometric-feedback results, one identical configuration across four visual modalities (Sonnet 4, $T$=0, 3 seeds, alignment-inclusive reward). SS/Rev. are mean defects per artifact, single-shot vs. reviewed; Δ is the mean defect reduction; CI is a paired bootstrap 95% interval on Δ% (10k resamples); $p$ is a two-sided paired sign test over decisive (task, seed) pairs. Web is summed over 1920/375 widths, animations over sampled frames.

| Modality | $n$ | SS | Rev. | Δ | 95% CI | impr./regr. ($p$) |
|---|---|---|---|---|---|---|
| Academic figures (20) | 60 | 0.57 | 0.15 | −74% | [52, 89]% | 17/2 ($7\times10^{-4}$) |
| Dense slides (12) | 36 | 1.25 | 0.33 | −73% | [45, 93]% | 13/1 ($1.8\times10^{-3}$) |
| Web pages (20) | 60 | 16.1 | 8.5 | −47% | [30, 62]% | 33/2 ($4\times10^{-8}$) |
| Animations (20) | 60 | 16.9 | 10.2 | −40% | [10, 64]% | 34/7 ($3\times10^{-5}$) |

### B.6 Cross-model replication (code)

**Table 7.** Cross-model replication on the same 15 hard code tasks, each family over *three* independent on-policy seeds at $T$=0.7 (Claude is the multi-run reference from Table 8); underlying Figure 8. The reviewed ceiling has zero seed variance for all three families, and the harness lift survives across Anthropic, Alibaba, and OpenAI. Single-shot per-seed pass rates: Qwen 66.7/73.3/73.3%, GPT-5 86.7/80.0/73.3%.

| Model | Single-shot pass | Reviewed pass | Δ (lift) |
|---|---|---|---|
| Claude Sonnet 4 (3 seeds) | $66.7 \pm 6.7\%$ | $\mathbf{100.0 \pm 0.0}\%$ | $+\mathbf{33.3}$ pp |
| Qwen3-235B-Instruct-2507 (3 seeds) | $71.1 \pm 3.8\%$ | $\mathbf{93.3 \pm 0.0}\%$ | $+\mathbf{22.2}$ pp |
| GPT-5 (3 seeds) | $80.0 \pm 6.7\%$ | $\mathbf{100.0 \pm 0.0}\%$ | $+\mathbf{20.0}$ pp |

### B.7 Held-out generalization (code)

**Table 8.** Held-out generalization on 32 newly constructed code tasks (zero overlap with the 15-task development set). The harness recovers 3/3 single-shot failures and regresses zero passing tasks. The smaller absolute Δ is a baseline-compression effect (single-shot is 90.6% on the held-out set, leaving at most 9.4 pp headroom).

| Split | $N$ | Single-shot | Reviewed | Δ | SS-fail fixes / regr. |
|---|---|---|---|---|---|
| Development (3-run mean) | 15 | $66.7 \pm 6.7\%$ | $100.0 \pm 0.0\%$ | $+33.3$ pp | 5/5 / 0 |
| **Held-out v2** | 32 | 90.6% | **100.0**% | $+\mathbf{9.4}$ pp | **3/3 / 0** |

### B.8 Budget-allocation simplex

Sweeping how a fixed token budget is split between proposer, execution, and reviewer produces an enormous spread in pass rate (Figure 15), rising monotonically with the proposer's share: review-heavy corners collapse almost to zero, while propose-heavy splits plateau at the top. The best split is *propose-heavy* (give most of the budget to generation, reserving just enough for the reviewer to locate defects); extra compute is better spent on more samples than on deeper iteration past the early-stop point.

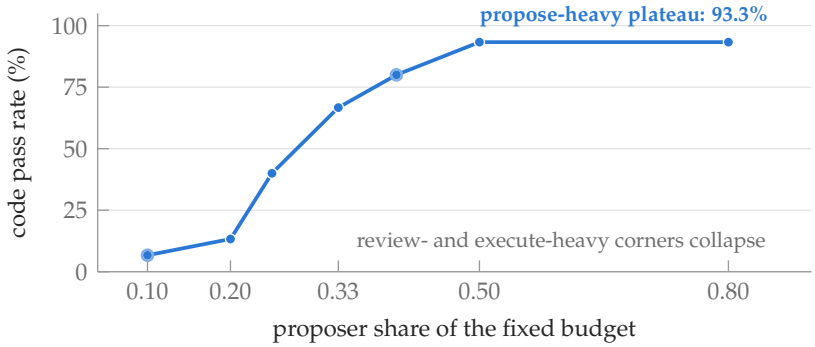



**Figure 15. Pass rate is monotone in the proposer's budget share.** Nine allocations of a fixed 10K-token budget across proposer / execution / reviewer, on the hard code suite (open markers: a second allocation with the same proposer share but a different execution/review split, showing that the proposer share alone predicts the outcome). The spread across the sweep is 86.6 pp; full simplex in Table 9.

**Table 9.** Budget allocation simplex on 15 hard code tasks, $B = 10$K total; underlying Figure 15. Pass rate is monotone in the proposer's per-call cap, and review-heavy corners (low $b_1$) collapse. The 9-point sweep yields an 86.6 pp spread.

| Allocation | $b_1$ (prop) | $b_2$ (exec) | $b_3$ (rev) | Pass rate | Mean tokens |
|---|---|---|---|---|---|
| **A (propose-heavy)** | 0.80 | 0.10 | 0.10 | **93.3% (14/15)** | 3,101 |
| **G (prop-dominant)** | 0.50 | 0.25 | 0.25 | **93.3% (14/15)** | 2,697 |
| D (prop+exec) | 0.40 | 0.40 | 0.20 | 80.0% (12/15) | 4,025 |
| E (prop+review) | 0.40 | 0.20 | 0.40 | 80.0% (12/15) | 4,388 |
| I (equal) | 0.33 | 0.34 | 0.33 | 66.7% (10/15) | 4,845 |
| H (review-dominant) | 0.25 | 0.25 | 0.50 | 40.0% (6/15) | 7,485 |
| F (exec+review) | 0.20 | 0.40 | 0.40 | 13.3% (2/15) | 8,771 |
| B (execute-heavy) | 0.10 | 0.80 | 0.10 | 6.7% (1/15) | 9,383 |
| C (review-heavy) | 0.10 | 0.10 | 0.80 | 6.7% (1/15) | 9,383 |
| | | | Spread: | **86.6 pp** | |

### B.9 Token ROI by modality

**Table 10.** Per-task token cost and quality return-on-investment for the harness, averaged across three on-policy runs. "Extra tokens" is reviewed-minus-single-shot per task. Tokens per 0.01 quality is extra tokens divided by (mean reviewed quality − mean single-shot quality)·100. Code dominates by 100×.

| Modality | Single-shot tokens | Reviewed tokens | Extra tokens | Tokens / 0.01 quality |
|---|---|---|---|---|
| Code | 3,336 | 4,575 | 1,239 | **37** |
| Video | 3,808 | 20,584 | 16,776 | 359 |
| Research | 14,116 | 27,793 | 13,677 | 2,415 |
| Webpages | 14,060 | 80,148 | 66,088 | 5,128 |
| Animations | 25,491 | 89,009 | 63,518 | 4,331 |
| Slides | 10,274 | 36,956 | 26,682 | 4,447 |

### B.10 Distillation: headline

**Table 11.** Distilled 8B student on the 18-task hard held-out suite (teacher's pre-curated single-shot failures, so teacher single-shot is 0/18 by construction). The student lifts judge-keep from 0% to 44% at pass@1 and 56% at pass@3. The 0.51× mean@1 capability ratio against the teacher is established on a separate 44-task out-of-distribution suite (Table 12), where teacher single-shot is non-degenerate.

| Setup | Compute | Judge-keep |
|---|---|---|
| 1 sample × 5 turns | 1× | 44% (8/18) |
| 1 sample × 8 turns | 1.6× | 28% (regressed) |
| **3 samples × 5 turns (pass@3)** | 3× | **56% (10/18)** |

### B.11 Distillation: out-of-distribution capability ratio

**Table 12.** Distilled 8B student (V3 + force-close) on a 44-task out-of-distribution suite where the teacher's single-shot is non-degenerate (teacher 20/44). Capability ratio is the student's judge-keep divided by the teacher's 20/44. Two sampling seeds are used, and pass@2 is the union over both. This is the suite that establishes the ∼0.51× mean@1 / 0.70× pass@2 ratios quoted in Section 7 and Figure 13.

| Metric | sample 1 | sample 2 | mean@1 | pass@2 |
|---|---|---|---|---|
| Judge-keep | 9/44 (20%) | 11/44 (25%) | 23% | **32**% |
| Capability ratio (vs teacher 20/44) | 0.45× | 0.55× | 0.51× | **0.70**× |

### B.12 Distillation: capability U-curve over force-close budget

**Table 13.** Reasoning-cap U-curve on the 18-task hard held-out suite. SFT recipe sweep on Qwen3-VL-8B-Instruct (37 judge-kept teacher traces, same inference protocol). The optimum is at the student's coherence horizon, not the teacher's.

| Variant | Reasoning cap (chars) | Judge-keep | Identical-retry |
|---|---|---|---|
| V1 (no reasoning) | 0 | 6% | 67% |
| V4 | 1000 | 11% | 35% |
| V2 | 3000 | 17% | 32% |
| **V3 (chosen)** | **1500** | **44%** | 10% |

### B.13 Distillation: data-scale

**Table 14.** Three independent attempts to scale the SFT corpus past V3's 37 examples. All regressed. Quality dominates quantity at this scale.

| Variant | Composition | $n$ examples | Judge-keep |
|---|---|---|---|
| **V3 (chosen)** | 235B teacher, judge-kept | 37 | **44%** |
| V5 | V3 + 13 judge-rejected (235B) | 50 | 33% |
| V6 | V3 + 20 judge-kept (30B teacher) | 57 | 33% |
| RFT v1 | Self-rolled, judge-filtered | 61 | 39% |

### B.14 Distillation: RFT regresses pass@3

**Table 15.** RFT polishes consistency at the cost of pass@$k$; underlying Figure 14. Per-turn metrics improve, per-task variance collapses, and pass@3 regresses by 17 pp. For deployment with sampling, the model's variance *is* the inference-scaling budget. *Note on notation.* **mean@1** is the expected pass rate of a single random sample, computed as the mean accuracy across the three sampling seeds (one $T$=0 greedy + two $T$=0.7). This is the natural variance-sensitive metric and differs from the greedy single-seed *pass@1*=44% reported in Table 11, which is just seed 1's accuracy. *pass@2* and *pass@3* are union pass rates over the first 2 / all 3 seeds.

| | Consistency metrics | | | Sampling-aware judge-keep | | |
|---|---|---|---|---|---|---|
| | id-retry ↓ | addresses-rev ↑ | no-artifact ↓ | mean@1 | pass@2 | pass@3 |
| **V3 (chosen)** | 9% | 83% | 2 | **31%** | **50%** | **56%** |
| RFT v1 | **8%** | **89%** | **1** | 28% | 39% | 39% |

### B.15 Per-task pass@$k$ for the 8B markdown student

**Table 16.** Per-task judge-keep outcome for the 8B markdown student on the 18-task hard held-out suite. ✓ = judge-kept, · = judge-rejected.

| Category | Task | seed 1 ($T$=0) | seed 2 ($T$=0.7) | seed 3 ($T$=0.7) | pass@3 |
|---|---|---|---|---|---|
| webpages | web_002 | ✓ | · | · | ✓ |
| | web_003 | ✓ | · | ✓ | ✓ |
| | web_005 | · | · | · | · |
| | web_008 | · | · | · | · |
| | web_012 | · | · | · | · |
| slides | slide_001 | · | · | · | · |
| | slide_006 | · | · | · | · |
| | slide_007 | · | · | · | · |
| | slide_008 | ✓ | · | · | ✓ |
| | slide_010 | · | · | · | · |
| | slide_014 | ✓ | ✓ | ✓ | ✓ |
| | slide_018 | ✓ | ✓ | · | ✓ |
| | slide_020 | · | ✓ | · | ✓ |
| code | code_h001 | ✓ | · | · | ✓ |
| | code_h002 | ✓ | ✓ | · | ✓ |
| | code_h003 | · | · | · | · |
| | code_h004 | · | · | ✓ | ✓ |
| | code_h005 | ✓ | ✓ | ✓ | ✓ |
| | Total kept: | 8/18 | 5/18 | 4/18 | **10/18 (56%)** |